Visual Basic---

Introduction

Visual Basic (VB) is an event-driven programming language and environment from Microsoft that provides a graphical user interface (GUI) which allows programmers to modify code by simply dragging and dropping objects and defining their behavior and appearance. VB is derived from the BASIC programming language and is considered to be event-driven and object-oriented.

VB is intended to be easy to learn and fast to write code with; as a result, it is sometimes called a rapid application development (RAD) system and is used to prototype an application that will later be written in a more difficult but efficient language.

The last version of VB, Visual Basic 6, was released in 1998, but has since been replaced by VB .NET, Visual Basic for applications (VBA) and Visual Stuido .NET. VBA and Visual Studio are the two frameworks most commonly used today.

Visual Basic features and characteristics

VB is a GUI-based development tool that offers a faster RAD than most other programming languages. VB also features syntax that is more straightforward than other languages, a visual environment that is easy to understand and high database connectivity.

Visual Basic was designed to be a complete programming language that contained ordinary features, such as string processing and computation. The visual environment is characterized by a drag-and-drop feature which allows programmers to build a user interface that is easy to use, even for developers with minimum experience.

While these features of VB are advantageous, there are others that can have a negative effect. The VB programming environment requires a large amount of memory, both for the initial installation and to run efficiently afterwards. The graphical features of the programming tool take up a large amount of space and require a significant amount of memory.

Furthermore, Visual Basic is not useful when developing programs that require a lot of processing time, like games, and the use of VB is restricted to Microsoft operating systems (OS).

Finally, with C languages, programmers can feasibly locate and use the defined values for variable data in a computer program at declaration time.
This initialization practice is something that isn't easily done with VB.

How Visual Basic is used

The structure of VB is designed to allow programmers to use the environment to write executable files (exe files). Also, using VB, developers can create programs that can be utilized as a front end to databases. VB tools can help programmers develop applications or complete software while still allowing them to modify and revise their work accordingly.

The most popular type of Visual Basic in use today is VBA. VBA is a version of Visual Basic that can be used to program Microsoft Office apps, such as Excel and PowerPoint. However, it can only be used to modify existing apps; VBA cannot be used to create new apps.

Benefits of Visual Basic

The BASIC programming language, which VB is derived from, is simple and easy to work with, especially when writing exe files.

However, VB becomes extremely beneficial when used with Microsoft's COM interface. The COM components can be written in various languages and then integrated using VB. Additionally, VB provides not only a programming language, but an integrated development environment (IDE) that has been written and optimized to best support RAD. This allows programmers to easily build GUIs and connect them to functions within the application.

Overall, VB enables the rapid development of Windows based applications while also assisting in the access of databases by using ActiveX data objects (ADO) while allowing programmers to use ActiveX control and various objects.

Difference between Visual Programming and Non Visual Programming

Visual Programming	Non-Visual Programming
Visual Programming requires **IDE**: Integrated Development Environment.	Non Visual Programming does not require IDE : **Integrated Development Environment**.
Visual Programming uses IDE so development of programme can be in done **fast** even though you don’t know exact language **syntax.**	In Non Visual Programming, there is no IDE, so developer has to be master in language syntax & fast typing as well for the fast development.

Visual Programming provides **visual expression, highlights errors,** suggest available methods and properties.	Non Visual Programming does not provide such features.
In Visual Programming, you can create programme, by using elements graphically i.e. **GUI**.	In Non Visual Programming, you have to create programme textually, no GUI is there, only **Text based interface.**
With Visual Programming, you can drag drop programme elements, can draw, can click, use menus, forms, dialogue boxs etc.	With Non Visual Programming, You have to write down programme in the form of text only.
Visual programming helps non-programmers to involved in program/logic easily.	Non Visual programming is difficult but also increase you memory power and **logic**.

Differences between Procedural and Object Oriented Programming

Procedural Programming

Procedural Programming can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions, simply consist of a series of computational steps to be carried out. During a program's execution, any given procedure might be called at any point, including by other procedures or itself.

Object-Oriented Programming

Object-oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object-oriented programming, computer programs are designed using the concept of objects that interact with the real world. Object-oriented programming languages are various but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

Languages used in Object-Oriented Programming:
Java, C++, C#, Python,

PHP, JavaScript, Ruby, Perl,

Objective-C, Dart, Swift, Scala.

Procedural Programming vs Object-Oriented Programming

Below are some of the differences between procedural and object-oriented programming:

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#,

Event-Driven Programming

Event-driven programming is a great approach for building complex systems. It embodies the divide-and-conquer principle while allowing you to continue using other approaches like synchronous calls.

When discussing event-based systems, several different terms often refer to the same concept. For simplicity, we'll primarily use the terms listed below in bold:

- **Event**, message, and notification
- **Producer**, publisher, sender, and event source
- **Consumer**, receiver, subscriber, handler, and event sink
- **Message queue** and event queue

What is event-driven programming?

Event-driven programming, or event-oriented programming, is a paradigm where entities (objects, services, and so on) communicate indirectly by sending messages to one another through an intermediary. The messages are typically stored in a queue before being handled by the consumers.

Unlike in using direct calls, event-driven programming completely decouples the producer from the consumer, leading to some noteworthy benefits. For example, multiple producers and multiple consumers can all collaborate to process incoming requests. Retrying failed operations and maintaining an event history are also simplified. Event-driven programming also makes it easier to scale large systems, adding capacity simply by adding consumers.

Let's look at the actors of event-based systems: events, producers, consumers, and message queues.

What are events?

Events are pieces of data that are sent by producers and eventually consumed by consumers. For example, a mouse down event typically contains the following information:

- Mouse pointer coordinates
- Which mouse buttons are pressed

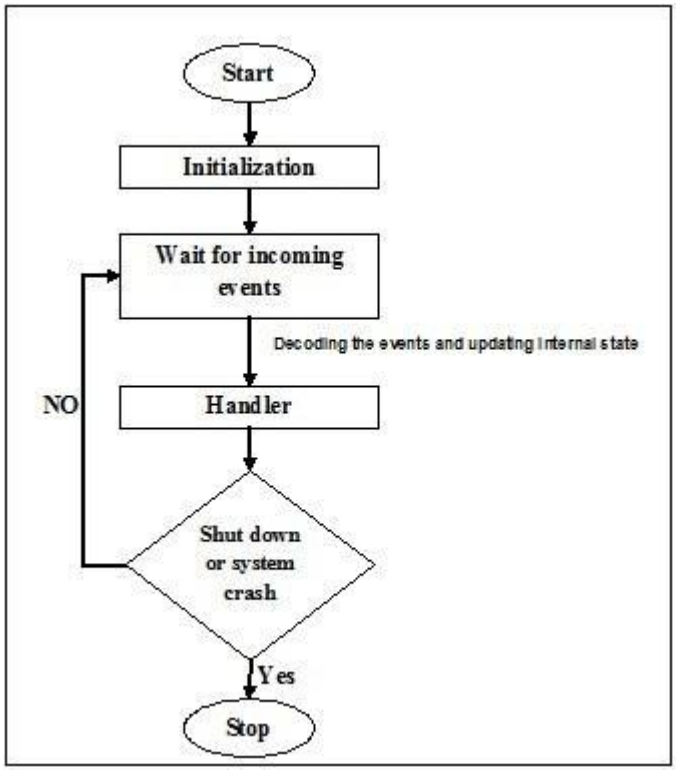
Events are usually structured but can sometimes just be blobs of data (such as a chunk of JSON) that consumers should know how to parse and handle.

Event-Driven Programming vs. Object-Oriented Programming

Events can be user-generated events that arise due to user actions, system events in the operating system, and software events generated by the program. Event-driven programming is a development of the ideas of top-down design when the reactions of the program to various events are gradually determined and detailed.

Object-oriented programming brings together the best ideas embodied in structured programming and combines them with powerful new concepts to optimally organize programs. Object-oriented programming allows you to decompose a problem into its parts. Each component becomes an independent object containing its codes and data that relate to this object. In this case, the whole procedure is simplified, and the programmer can operate with much larger programs.

Event-driven programming focuses on events. Eventually, the flow of program depends upon events. Until now, we were dealing with either sequential or parallel execution model but the model having the concept of event-driven programming is called asynchronous model. Event-driven programming depends upon an event loop that is always listening for the new incoming events. The working of event-driven programming is dependent upon events. Once an event loops, then events decide what to execute and in what order. Following flowchart will help you understand how this works –



The event loop

Event-loop is a functionality to handle all the events in a computational code. It acts round the way during the execution of whole program and keeps track of the incoming and execution of events. The Asyncio module allows a single event loop per process. Followings are some methods provided by Asyncio module to manage an event loop −

- **loop = get_event_loop()** − This method will provide the event loop for the current context.
- **loop.call_later(time_delay,callback,argument)** − This method arranges for the callback that is to be called after the given time_delay seconds.
- **loop.call_soon(callback,argument)** − This method arranges for a callback that is to be called as soon as possible. The callback is called after call_soon() returns and when the control returns to the event loop.
- **loop.time()** − This method is used to return the current time according to the event loop's internal clock.
- **asyncio.set_event_loop()** − This method will set the event loop for the current context to the loop.
- **asyncio.new_event_loop()** − This method will create and return a new event loop object.

- **loop.run_forever()** – This method will run until stop() method is called.

Example

The following example of event loop helps in printing **hello world** by using the get_event_loop() method. This example is taken from the Python official docs.

```
import asyncio

def hello_world(loop):
   print('Hello World')
   loop.stop()

loop = asyncio.get_event_loop()

loop.call_soon(hello_world, loop)

loop.run_forever()
loop.close()
```

Output

```
Hello World
```